

RPL：面向高频机器人通信的 C++ 低延迟协议解析与无锁状态同步库

梁家祥

大连民族大学 C · ONE 战队

编译日期: 2026 年 4 月 9 日

摘要

我们提出了 RPL (RoboMaster Packet Library), 一个从零开始构建的、兼顾极致性能与内存安全的现代 C++20 序列化与通信库。在复杂的机器人控制系统（如 RoboMaster 赛事）中，微控制器与协处理器之间的高频数据交互对延迟和内存连续性有着极其苛刻的要求。RPL 采用 BipBuffer 解决传统环形缓冲区（RingBuffer）跨界内存不连续问题，并利用编译期编程构建了 O(1) 复杂度的协议路由表与连续内存池映射。此外，RPL 实现了编译期位域解析，将位段的偏移量与掩码值在编译期固化，运行时无需查表或分支判断。在架构设计上，RPL 通过 SeqLock（顺序锁）实现了严格的“读写解耦”，确保了数据校验的绝对安全与最优的 CPU 内存对齐访问。性能测试表明，在 Intel i7-1185G7 平台上，RPL 的数据包反序列化耗时 **0.526 ns**，小包解析（16B）耗时 **75 ns**（含 CRC8 + CRC16 校验），序列化耗时 **21.2 ns**，达到了极高的工程水准。

关键词：C++20, 机器人通信, 编译期编程, 序列化, RoboMaster

代码元数据

属性	描述
当前代码版本	v1.0.0
开源仓库链接	github.com/RoboMaster-DLMU-CONE/rpl
开源协议	ISC License
编程语言	C++20
编译依赖	无 (Header-Only)

1. 引言

在现代机器人系统（特别是 RoboMaster 等高强度竞技机器人）中，底层硬件（MCU）与上层视觉/导航系统（MiniPC）之间的通信频率通常高达数百至数千赫兹。传统的通信协议栈在处理这种高频字节流时，往往面临着严重的性能瓶颈和内存碎片问题。与 ROS 等成熟的机器人操作系统 [2] 相比，在资源受限的嵌入式环境中，开发者需要更轻量、更确定的通信方案。

目前广泛使用的标准做法是利用环形缓冲区（RingBuffer）结合 DMA 接收数据。然而，传统的 RingBuffer 经常遭遇“数据包截断”问题——即一个完整的数据帧被物理分割在缓冲区的尾部和头部。这迫使开发者必须编写繁琐的逻辑进行拼接，或进行额外的内存拷贝操作，导致 CPU 资源的浪费。此外，传统协议解析大量依赖运行时的动态计算与查表，进一步增加了系统的不可预测性。

本文提出的 RPL (RoboMaster Packet Library) 通过引入 Bip-Buffer 技术和现代 C++ 编译期操作，彻底解决了上述问题。Bip-Buffer 的概念最初由 Cooke[1] 提出，它通过巧妙的预留与翻转机制，保证了即使在缓冲区发生回绕时，也能向解析器提供一段绝对连续的可用内存空间。本库的设计由以下三个核心原则指导：

- 极致性能** 将一切可能的计算（结构体映射、哈希、位域掩码生成）转移到编译期完成。
- 绝对安全与对齐** 拒绝污染业务内存，确保交付给应用层的数据不仅通过了严格校验，且符合最优的 CPU 内存对齐标准。

- 极简接口** 业务逻辑只需通过一次 `get()` 函数调用，即可 O(1) 获取最新状态，无需编写复杂的回调函数。

2. 核心架构与特性

RPL 的架构设计可以分为“数据流接收缓冲区”与“编译期内存池”两个独立且协同的模块。

2.1. BipBuffer 与两段式数据流处理

为了解决传统 RingBuffer 的内存不连续问题，底层 DMA 接收模块选用了 BipBuffer。BipBuffer 通过巧妙的预留与翻转机制，保证了即使在缓冲区发生回绕时，也能向解析器提供一段绝对连续的可用内存空间。

更重要的是，RPL 在处理“网络字节流”到“业务数据”的转换时，采取了“**BipBuffer 解析** → **校验** → **内存池拷贝**”的两段式架构。虽然直接使用 `reinterpret_cast` 在 BipBuffer 中原地零拷贝地映射数据听起来具有诱惑力，但这在工程实践中存在隐患。与 FlatBuffers[4] 等业界著名的“零拷贝”反序列化框架不同，RPL 选择将解析后的 Payload 拷贝至独立内存池的原因有三：

- 彻底的安全性**：如果数据包的 CRC 校验失败，该包将在 Bip-Buffer 阶段被直接丢弃，绝对不会进入/污染上层的业务内存池。
- 业务极大的便利性**：数据池扮演了“黑板”的模式。上层业务线程（如姿态解算、自瞄逻辑）可以随时随地调用 `get()` 获取最新状态，实现了读写解耦，免去了复杂且易造成死锁的回调机制。
- 完美的内存对齐**：网络传输的字节流往往是紧凑且未对齐的，直接在错位内存上进行访问会导致 CPU 性能下降甚至触发硬

作者邮箱: moonfeather120@outlook.com

本文介绍了 RPL (RoboMaster Packet Library) 的设计理念、核心架构与性能测试结果。该库为机器人通信领域提供了一个高性能、内存安全的现代 C++20 序列化解决方案。

件异常。而拷贝至内存池的结构体是严格遵循 C++ 标准对齐的，保证了后续业务逻辑最高的 CPU 访问效率。

读写解耦的核心是 **SeqLock**（顺序锁）机制：每次写入内存池前后，`Deserializer` 会递增一个版本号（奇数表示写入中，偶数表示写入完成）。业务线程通过 `get<T>()` 读取时，仅需检查版本号是否在读取过程中保持一致且为偶数。该机制无需互斥锁（Mutex），避免了读线程被写线程阻塞的风险。SeqLock 是 Linux 内核中经典的无锁并发原语 [3]。在单核嵌入式平台上，SeqLock 退化为 `volatile` + 编译器屏障，开销极低；在 SMP 多核环境下可通过 `RPL_USE_STD_ATOMIC` 宏切换为 `std::atomic`，保证跨核可见性。

2.2. 编译期路由与连续内存池映射

RPL 对多协议数据流的处理分为解析路由和反序列化入池两个阶段，两阶段均依赖编译期计算，避免了运行时的动态分配与哈希碰撞。

第一阶段：解析路由（Parser → header_lut）

在 `Parser<PacketA, PacketB, ...>` 实例化时，RPL 会在编译期扫描所有注册数据包的 `Protocol::start_byte`，自动生成一张 256 字节的直接查找表（`header_lut`）。该表将每个可能的起始字节直接映射到对应的协议 Worker 索引：

```
// 编译期生成（伪代码示例）
static constexpr std::array<uint8_t, 256> header_lut = []() {
    std::array<uint8_t, 256> table{};
    table.fill(0xFF); // 0xFF 表示未注册
    table[PacketA::Protocol::start_byte] = 0; // Worker 0
    table[PacketB::Protocol::start_byte] = 1; // Worker 1
    // ...
    return table;
}();
```

代码 1. 编译期路由表生成示例代码

解析时，Parser 只需读取接收到的字节 `header_lut[byte]`，即可 $O(1)$ 获得对应的协议 Worker，随后调用该 Worker 执行帧头校验、分段 CRC 验证和 Payload 提取。由于路由表在编译期完全确定，整个过程无需运行时查表或分支判断。

此外，当多个数据包的 `start_byte` 发生冲突时，`header_lut` 的构建过程会触发 `RPL_ERROR_START_BYTE_COLLISION()` 诊断函数，将问题提前到编译期暴露，避免运行时的解析歧义。

第二阶段：反序列化入池（Deserializer → frozen 编译期哈希）

当数据包通过 CRC 校验后，Parser 将 Payload 分段拷贝至 `Deserializer`。反序列化器内部维护了一块连续对齐的内存池（`MemoryPool`），池中为每个数据包类型预分配了固定偏移。

为了将帧中的 `cmd_id` 快速映射到内存池偏移量，RPL 借助 frozen 库 [6] 在编译期构造了 `frozen::unordered_map<uint16_t, size_t>`。该哈希表的键值对（cmd 到 offset 的映射）在编译期完全确定，查找过程在运行时仅需数条指令即可完成。写入时，RPL 使用 **SeqLock**（顺序锁）保护内存池，确保多线程读取时的数据一致性：

```
// 编译期映射（伪代码示例）
```

```
static constexpr auto cmdToIndex = frozen::make_unordered_map({
    {PacketA::cmd, offset_A},
    {PacketB::cmd, offset_B},
    // ...
});
```

代码 2. 编译期命令到偏移量映射

整个“解析 → 校验 → 入池”流程无需任何运行时注册或动态内存分配，所有路由和布局信息均在编译期固化，达到了极致的确定性延迟。

2.3. 编译期位域解析

在机器人协议中，为了节省带宽，常常将多个布尔值或小整数压缩在一个字节（位域）中。传统的位域解析依赖运行时的位掩码运算，代码冗长且容易出错。RPL 提供了基于模板元编程的位流解析引擎，将位域的布局信息完全转移到编译期描述。现代 C++ 模板元编程的理论与实践在 Vandevoorde 等人的著作 [5] 中得到了系统阐述。

用户通过 `std::tuple<Field<T, Bits>...>` 在数据包定义中声明每个位段的类型和宽度：

```
struct GameStatus {
    uint8_t game_type;
    uint8_t game_progress;
    uint16_t stage_remain_time;
    // ...
    using BitLayout = std::tuple<
        RPL::Meta::Field<uint8_t, 4>, // game_type: 4 bits
        RPL::Meta::Field<uint8_t, 4>, // game_progress: 4 bits
        RPL::Meta::Field<uint16_t, 16> // stage_remain_time: 16
    > bits;
    // ...
};
```

代码 3. 位域布局定义示例

在运行时，`deserialize_bitstream<T>()` 函数会根据编译期布局自动执行以下操作：

1. 前缀和计算：编译期生成每个位段在字节流中的起始偏移量（`offsets` 数组），无需运行时累加。
2. 跨字节提取：通过 `extract_bits<T, BitOffset, BitWidth>()` 模板函数处理位段的边界跨越问题。由于 `BitOffset` 和 `BitWidth` 均为编译期常量，编译器在 `-O3` 优化下可将移位、掩码等操作折叠为常量指令，CPU 只需执行最基础的 `»`、`\&`、`|` 运算，无需查表或分支判断。
3. 结构化重组：借助 C++20 的 `std::make_from_tuple`，将提取出的值直接聚合初始化为目标结构体。

序列化过程同理，通过 `inject_bits<T, BitOffset, BitWidth>()` 将结构体成员的值按编译期布局打包回字节流。实测位域序列化仅需 **13.7 ns** (x86)，证明了该元编程引擎生成的指令极为精简。

2.4. 可选的连接健康检测

在机器人系统中，通信链路可能因线缆松动、串口故障或电磁干扰而中断。RPL 提供了一套编译期策略模式的连接健康检测机制，使开发者能够在不引入运行时开销的前提下感知通信异常。

Parser 模板支持将 `ConnectionMonitor` 作为首个模板参数。RPL 内置了三种实现：

表 1. 连接监控器类型对比

监控器类型	说明	开销
<code>NullConnectionMonitor</code>	默认实现，所有方法为空	零（编译器完全优化掉）
<code>TickConnectionMonitor</code>	基于时间戳的超时检测	每次解析记录时间戳
<code>CallbackConnectionMonitor</code>	用户自定义回调	由回调函数决定

```
// 1. 定义时间戳提供者（适配任意平台）
struct HALTickProvider {
    using tick_type = uint32_t;
    static tick_type now() { return HAL_GetTick(); } // STM32
    ↪ HAL 库
};

// 2. 实例化带监控的 Parser
using Monitor = RPL::TickConnectionMonitor<HALTickProvider>;
RPL::Parser<Monitor, GameStatus, ChassisControl> parser{
    ↪ deserializer};

// 3. 业务循环中检查连接状态
void control_loop() {
    if (!parser.get_connection_monitor().is_connected(100)) {
        // 超过 100ms 未收到任何数据包，执行降级策略
        enter_safe_mode();
    }
    // ...
}
```

代码 4. 连接健康检测使用示例

用户只需提供一个符合 `TickProviderConcept` 的时间戳源（如 STM32 的 `HAL_GetTick()`、Linux 的 `clock_gettime()`），即可实现毫秒级的连接超时检测。当不需要监控功能时，默认的 `NullConnectionMonitor` 会被编译器完全优化为空的函数调用，运行时开销为零。

3. 典型应用场景

RPL 的设计初衷不仅是为了追求极致的性能，更是为了在实际的工程开发中极大降低开发者的心智负担。以下是 RPL 在现代机器人系统中的几个典型应用场景：

3.1. 上下位机协同与图形化工具链

在现代机器人架构中，通常由 MCU（如 STM32）负责底层电机实时控制，由协处理器（MiniPC，如 Jetson/NUC）负责视觉识别与路径规划。双端的通信协议通常需要手动编写冗长的结构体

定义和打包/解包代码，这不仅耗时，且在协议变更时极易发生双端结构不一致的致命错误。

为了解决这一痛点，RPL 提供了一套“单一数据源”的图形化工具链。开发者只需在 Web 端图形界面中定义位段与数据类型：

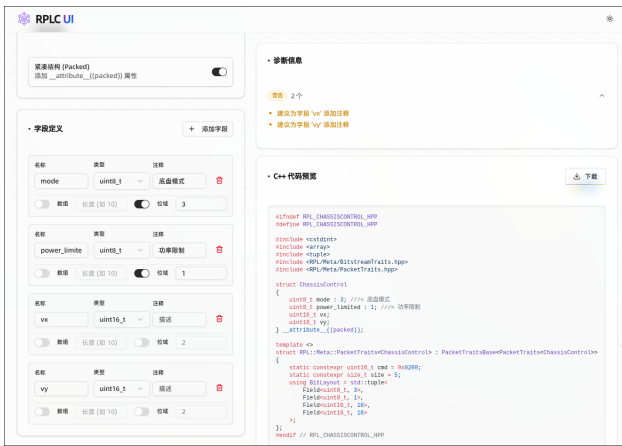


图 1. RPL Web 端在线协议生成器（支持位域、数组分配与冲突检测）

工具会自动所见即所得地生成出符合 RPL 规范的 `.hpp` 头文件。MCU 和 MiniPC 只需同时 `#include` 该文件，即可直接实现跨平台的无缝通信。RPL 屏蔽了内存对齐差异，彻底消除了手动编写序列化代码的繁琐与潜在 Bug。

3.2. 复杂官方协议的原生直连解析

以 RoboMaster 赛事为例，官方提供的“裁判系统”串口数据流包含了机器人血量、弹量、比赛倒计时等数十种不同格式的复杂数据包。传统的处理方式需要开发者手写庞大的 `switch-case` 状态机来逐字节匹配帧头和命令码，代码极其臃肿且难以维护。

借助 RPL，开发者（或开源社区）只需将裁判系统的各项协议定义为一系列 `PacketTraits`。在运行时，无论裁判系统的数据包以何种顺序、何种碎片化程度到达串口，只需一股脑地将其推入 Parser，RPL 的编译期路由表会自动完成 $O(1)$ 的包体分发与 CRC 校验，将各项状态精准映射到内存池中。业务层只需按需调用 `get<GameStatus>()`，即可获得绝对安全且最新的比赛数据。

3.3. 传输介质无关的“纯字节流”抽象

许多传统的序列化库往往与特定的网络 Sockets 或串口驱动强绑定，难以移植。而 RPL 采用了极其干净的纯字节流抽象设计。

Parser 的输入仅仅是一段无状态的内存指针与长度。这意味着 RPL 对底层通信介质是完全解耦的：

- **串口 (UART/USART)：**配合 DMA 实现零拷贝接收。
- **USB (CDC / Bulk Endpoint)：**应对高吞吐量的视觉数据传输。
- **网络 (UDP / TCP)：**在多台机器人间的局域网图传与状态同步，甚至是跨网段的远程遥控。
- **CAN 总线：**在处理好分包/组包（如 ISO-TP）的前提下，RPL 同样可以作为上层的 Payload 解析引擎。

无论底层硬件如何变化，上层的 `Parser` 和 `Deserializer` 逻辑无需改动一行代码。

3.4. 多链路异构数据的统一聚合

在复杂的机甲系统中，数据往往来源于多个物理接口。例如：UART1 接收裁判系统，UART2 接收自研超级电容，USB 接收视觉自瞄数据。

得益于 RPL 的模板实例化设计，开发者可以轻松实现多数据源的聚合。可以实例化三个独立的 `Parser` 分别对接三种物理接口，但将它们共同绑定到同一个 `Deserializer` 内存池上。这样一来，内存池就像一个系统级的“全局状态黑板”。所有传感器与通信链路在底层并发写入各自的状态，而控制算法线程只需面对这一个黑板进行 `get()` 读取。这极大地简化了多传感器信息融合的代码架构。

4. 代码示例

RPL 旨在为开发者屏蔽底层的复杂性。用户可通过在线协议生成器图形化设计数据包结构并自动导出 `.hpp` 文件，无需手动编写 `PacketTraits` 特化。以下代码展示了如何定义一个数据包并进行解析与获取。

4.1. 定义数据包结构

```
#include <RPL/Packets/GameStatus.hpp> // 裁判系统预定义包
#include <RPL/Packets/RobotStatus.hpp>
#include <RPL/Deserializer.hpp>
#include <RPL/Serializer.hpp>
#include <RPL/Parser.hpp>

// 自定义数据包（使用位域压缩）
struct ChassisControl {
    uint8_t mode : 3; // 3 bits: 底盘模式
    bool power_limited : 1; // 1 bit: 功率限制
    uint16_t vx; // 2 bytes: X 轴速度
    uint16_t vy; // 2 bytes: Y 轴速度
    float omega; // 4 bytes: 角速度
} __attribute__((packed));

// 编译期位域布局描述
template <>
struct RPL::Meta::PacketTraits<ChassisControl> : PacketTraitsBase
{
    << PacketTraits<ChassisControl>> {
    static constexpr uint16_t cmd = 0x0200;
    static constexpr size_t size = 10;
    using BitLayout = std::tuple<
        RPL::Meta::Field<uint8_t, 3>, // mode: bits [0:2]
        RPL::Meta::Field<bool, 1>, // power_limited: bit [3]
        RPL::Meta::Field<uint16_t, 16>, // vx: bits
        << [16:31]
        RPL::Meta::Field<uint16_t, 16>, // vy: bits
        << [32:47]
        RPL::Meta::Field<float, 32> // omega: bits
        << [48:79]
    >;
    };
};
```

代码 5. 数据包结构定义示例

4.2. 初始化解析器与反序列化器

```
// 注册需要处理的所有数据包类型
RPL::Deserializer<GameStatus, RobotStatus, ChassisControl>
    << deserializer;
RPL::Parser<GameStatus, RobotStatus, ChassisControl> parser{
    << deserializer};
```

代码 6. 解析器与反序列化器初始化

4.3. 接收端：DMA / 串口中断回调

RPL 使用 `tl::expected<T, Error>` 替代异常机制进行错误处理，8 种错误码（`BufferOverflow`、`CrcMismatch`、`InvalidFrameHeader` 等）覆盖了所有可能的异常情况，适合禁用异常的嵌入式环境。

```
// 方式一：推模式 —— 每次收到数据调用 push_data()
void UART_IRQHandler(const uint8_t* rx_buf, size_t len) {
    auto result = parser.push_data(rx_buf, len);
    if (!result) {
        // 处理错误：缓冲区满、CRC 不匹配等
        // result.error().code -> RPL::ErrorCode::BufferOverflow
        // result.error().message -> 人类可读的描述
        log_error("Parse error: %s", result.error().message);
    }
}

// 方式二：零拷贝 DMA 直写 —— DMA 直接搬运到 BipBuffer 内部
void DMA_StartReceive() {
    auto span = parser.get_write_buffer(); // 获取连续可写区域
    HAL_UART_Receive_DMA(&uart1, span.data(), span.size());
}

// DMA 传输完成回调
void DMA_TransComplete(size_t received_len) {
    parser.advance_write_index(received_len); // 通知 Parser 解
    << 析
}
```

代码 7. 接收端 DMA/中断回调处理

4.4. 业务线程：获取最新数据

```
void control_loop() {
    // 0(1) 获取内存对齐后的最新状态，无需关心数据何时到达
    auto chassis = deserializer.get<ChassisControl>();
    auto game = deserializer.get<GameStatus>();

    // 直接使用，结构体成员已按 CPU 最优对齐
    if (chassis.mode == 2) { // 云台跟随模式
        chassis.vx = 100;
    }
}
```

代码 8. 业务线程获取数据示例

在 `ChassisControl` 结构体的定义中，我们使用了 `__attribute__((packed))` 保证网络字节流的紧凑布局。随后通过 `PacketTraits` 特化中的 `std::tuple<Field<T, Bits>...>` 声明了每个位段的类型和宽度，编译期即确定所有位移和掩码值。

在接收端中断回调(`DMA_TransComplete` 等)中,系统模拟了DMA将数据喂给 `BipBuffer` 后的处理流程。Parser 从 `BipBuffer` 获取连续读视图,执行帧头校验、CRC8/CRC16 校验,一旦验证通过,数据被分段拷贝入内存池。

在 `control_loop` 业务循环中,业务线程无需关心数据是何时到达的,仅需调用 `deserializer.get<ChassisControl>()`。该操作通过编译期生成的偏移量索引 $O(1)$ 定位到内存池对应位置,并由 `SeqLock` 保证读取一致性,直接返回内存对齐后的结构体副本。

5. 性能测试

为了验证 RPL 在不同硬件环境下的性能表现,我们进行了跨平台基准测试,涵盖了桌面级 x86 处理器和资源受限的嵌入式 ARM Cortex-M 内核。测试使用 Google Benchmark 框架 (x86) 和 Zephyr `k_cycle_get_32()` 周期计数器 (ARM),均开启最高速度优化级别。

5.1. x86 桌面平台

测试环境: 11th Gen Intel Core i7-1185G7 @ 4.80 GHz, Linux 系统, GCC 编译器, `-O3 -march=native`。

表 2. x86 平台性能测试结果

操作	延迟 (ns)	说明
反序列化 (Get)	0.526	从内存池提取已解析包, 仅需约 11 条指令
单包序列化 (Serialize)	21.2	包含帧头组装、CRC8 + CRC16 计算
位域序列化 (Bitfield)	13.7	编译期位域布局, 高效跨字节操作
小包解析 (Parse, 16B)	75.0	含包头搜索、CRC 校验、分段拷贝入池

5.2. 嵌入式 ARM Cortex-M3 平台

测试环境: Zephyr RTOS v4.3.0, QEMU Cortex-M3 模拟器, `CONFIG_SPEED_OPTIMIZATIONS=y`。

表 3. ARM Cortex-M3 平台性能测试结果

操作	周期数 (Cycles)	72 MHz 估算 (ns)	480 MHz 估算 (ns)
反序列化 (Get)	11	152	23
单包序列化 (Serialize)	158	2190	329
完整帧写入 (FullWrite)	434	6030	904
逐字节碎片写入 (1B Frag)	3948	54800	8200

- 嵌入式平台的测试结果揭示了两个重要实践指导:
- **DMA 至关重要:** 完整帧写入 (`FullWrite`) 比逐字节碎片写入快 9.1 倍。在 STM32 等微控制器上, 强烈建议配合 DMA 使用 `get_write_buffer()` 和 `advance_write_index()` 接口。
 - **编译器优化敏感:** 开启速度优化后, RPL 性能提升约 50%–60%。得益于深度模板化和内联友好的设计, 编译器能够激进地进行循环展开和寄存器分配。

5.3. 实际系统影响

上述数据表明, 即使在 1000 Hz (周期 1 ms, 即 1,000,000 ns) 的极高控制频率下, RPL 的小包解析耗时 (75 ns) 占比也仅为约十万分之一。在 480 MHz 的嵌入式平台上, 反序列化仅需 23 ns, 完全不会对任何高频实时控制回路产生可感知的阻塞。

Benchmark	Time	CPU	Iterations
BM_GetPacket	0.526 ns	0.526 ns	1000000000
BM_Serialize	21.2 ns	21.2 ns	33076923
BM_ParseSmall	75.0 ns	75.0 ns	9285714
BM_ParseXLarge	14490 ns	14490 ns	48571

代码 9. Google Benchmark 测试结果输出

6. 结语

RPL 证明了在资源受限且对延迟极其敏感的机器人通信领域, 现代 C++ 的编译期元编程与优秀的数据结构 (`BipBuffer`) 结合能爆发出巨大的潜力。通过在安全验证、业务解耦、内存对齐三者之间找到最佳的架构平衡, RPL 不仅提供了惊艳的性能指标, 也为 RoboMaster 及更广泛的机器人开发者提供了一个开箱即用的高性能网络协议库。

展望未来, RPL 将在以下方向持续演进:

- **更广泛的平台支持:** 完善 Arduino 生态集成, 探索对更多 RTOS (如 FreeRTOS、RT-Thread) 及 Linux 通信框架的原生适配。
- **协议兼容层:** 研究对 MAVLink 等通用机器人协议的兼容, 降低已有项目的迁移成本。
- **硬件在环测试:** 在真实 STM32 硬件上建立自动化测试管线, 进一步验证嵌入式环境下的确定性延迟表现。

RPL 的设计始终遵循一个朴素的目标: 让使用者少写代码, 让 CPU 少做无用功。我们相信, 好的库不仅跑得快, 更让人用得舒心。

致谢

本文章和 RPL 库送给热爱机甲大师的人们。
感谢织星在开发过程中提出的诸多宝贵意见。没有她的陪伴, 我不能完成这个项目。

感谢 C·ONE 战队所有成员的付出。特别感谢许佳琪队长为战队在各个方面做出的奉献。她的努力让我们能够有时间和机会探索优雅的新技术。特别感谢聂启翔学长, 他是我在战队中的引路人, 在本库的开发和调试过程中给予了很多宝贵的指引和意见。

参考文献

[1] S. Cooke, *The bip buffer - the circular buffer with a twist*, CodeProject, 2003. [Online]. Available: <https://www.codeproject.com/Articles/3479/The-Bip-Buffer-The-Circular-Buffer-with-a-Twist>

[2] M. Quigley et al., “ROS: An open-source robot operating system,” in *ICRA workshop on open source software*, vol. 3, 2009, p. 5.

- [3] R. Love, *Linux Kernel Development*, 3rd. Pearson Education, 2010.
- [4] Google, *Flatbuffers: Memory efficient serialization library*, 2014. [Online]. Available: <https://google.github.io/flatbuffers/>
- [5] D. Vandevorde, N. M. Josuttis.com, and D. Gregor, *C++ Templates: The Complete Guide*, 2nd. Addison-Wesley Professional, 2017.
- [6] serge-sans-paille et al., *Frozen: A header-only, constexpr alternative to gperf for c++14 users*, GitHub Repository, 2019. [Online]. Available: <https://github.com/serge-sans-paille/frozen>